The Bank Is Open: Predicting NBA Over/Under Totals with Machine Learning

UE24CS352A - Machine Learning, Mini-Project | Rohan (PES1UG24CS381), Sanupureddy Hemanth Kumar Reddy (PES1UG24CS418) | Code:https://github.com/rohancodexp/nba-overunder-ml

DEMO BUILD: every number below was produced on **synthetic** NBA-like data (the real-data sources were not reachable when this file was generated). Re-run `python run_all.py --data real` and `python scripts/build_report.py` to regenerate this write-up with real results.

1. Problem statement

Sportsbooks publish an **Over/Under** line: the expected combined points of both teams in an NBA game. We predict the **total points scored** in a game from information available before tip-off, minimise Mean Squared Error (MSE), and test whether the predictions can pick the correct side of the sportsbook line often enough to profit. At the standard -110 price a bettor must win **52.38%** of bets to break even. This follows the Stanford CS229 study The Bank Is Open: AI in Sports Gambling (Bucquet and Sarukkai), which reached 51.5% with a neural network.

2. Dataset

Source: Sportsbook Reviews Online (SBRO) NBA odds archive, one Excel file per season (2007-08 to 2017-18). Each game has the final score by quarter and the closing total line. **Split (chronological, never shuffled):** train 2012-13 to 2015-16 (n=4920), validation 2016-17 (n=1230) used only for tuning and early stopping, test 2017-18 (n=1230) evaluated once at the end. The sportsbook line is used **only for evaluation**, never as a feature. The first training season (2012-13) was itself tuned on validation data because NBA scoring drifts over time.

3. Approach

Features (46 numeric + 60 team one-hot), all computed from games strictly before the target game: for each team its last 3 games (points for/against, home/away, rest days, opponent season-to-date scoring and defence), current rest days, back-to-back flag, consecutive-away streak (a travel-fatigue proxy) and season-to-date averages. A unit test verifies there is no look-ahead leakage.
Models. (i) **Random Forest** baseline. (ii) **Collaborative filtering**: home team = user, away team = item, total points = rating; for each game day a sparse team x team matrix of the last h days of league results is factorised with regularised alternating least squares (masked SVD) and the prediction is $u_{home} \cdot v_{away}$. (iii) **Neural network** 500-100-20-1 with ReLU, Adam, L2 weight decay, early stopping. (iv) **LSTM** (written from scratch in NumPy with back-propagation through time, verified by a gradient-check test) that reads the last 3 games in order and combines its final state with current-context features.

Parameters vs. hyper-parameters. Parameters are learned by the algorithm: tree split points, the factor matrices U and V, network weights/biases, LSTM gate weights. Hyper-parameters are fixed before each fit and were tuned on the **validation season only** (Random Forest, NN and LSTM: random search; CF: full grid; every trial is logged in `results/tuning_log.csv`):

Model	Hyper-parameters tuned (search space)	Selected values
Random Forest	n_estimators, max_depth, min_samples_leaf, max_features	n_estimators=200, max_depth=8, min_samples_leaf=1, max_features=0.5
CF / SVD	n_factors {2,5,10}, horizon_days {3..30}, reg {0.3,1,3}	n_factors=10, horizon_days=3, reg=3.0
Neural Net	hidden sizes, learning rate, L2 alpha {1e-4..1}, batch size; epochs by early stopping	hidden=[500, 100, 20], lr=0.0003, alpha=1.0, batch_size=32; best epoch 24
LSTM	hidden size, learning rate, dropout, weight decay, batch size; epochs by early stopping	hidden=20, lr=0.01, dropout=0.0, weight_decay=0.001, batch_size=32; best epoch 1

4. Implementation overview

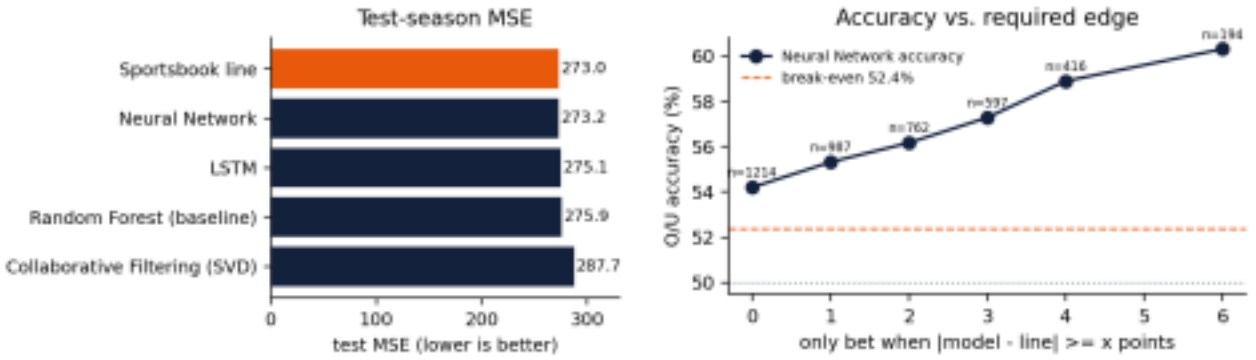
Python 3, scikit-learn, NumPy, pandas, matplotlib. `src/data_loader.py` reads SBRO files; `src/features.py` builds leakage-free features; `src/models/` holds the four models; `src/tuning.py` runs the validation-only search; `run_all.py` executes the whole pipeline with one command and writes metrics, figures and saved models; `demo.py` predicts games live; `tests/` has 5 automated tests. Seeds are fixed for reproducibility.

5. Results (test season, evaluated once)

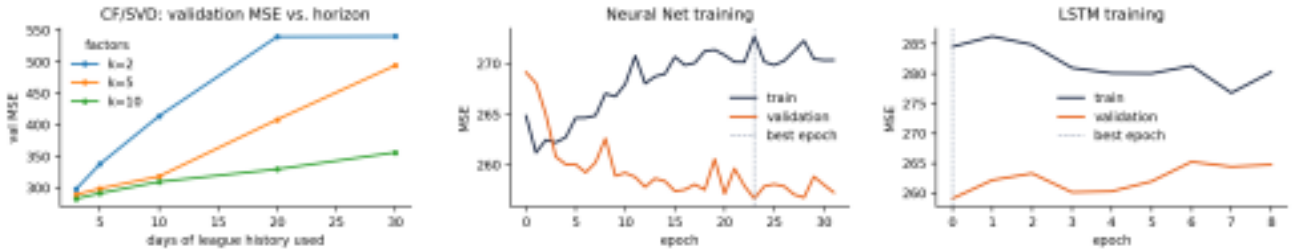
Model	Train MSE	Val MSE	Test MSE	Test MAE	O/U accuracy % (95% CI)	ROI at -110

Random Forest*	223.1	258.3	275.9	13.27	55.1 +/- 2.8	+5.2%
Collaborative Filtering (SVD)	306.6	282.6	287.7	13.57	52.9 +/- 2.8	+1.0%
Neural Network	272.7	256.6	273.2	13.22	54.2 +/- 2.8	+3.5%
LSTM	284.5	259.1	275.1	13.34	53.7 +/- 2.8	+2.5%
Sportsbook line	-	255.0	273.0	-	(reference)	-
Constant (train mean)	-	-	280.5	-	-	-

*baseline. O/U accuracy: bet OVER when the model total exceeds the line, UNDER otherwise; pushes excluded; CI is the binomial 95% interval.



Tuning and training evidence



Left: CF validation MSE against the history window (days of league results used). Middle and right: training and validation MSE per epoch; the dashed line marks the epoch with the lowest validation MSE, which is where early stopping restores the weights.

6. Conclusions

Best model (chosen on validation MSE): Neural Network, test MSE 273.2 versus 273.0 for the sportsbook line (+0.2); the model is effectively tied with the line on MSE. On Over/Under picks it scores 54.2% (95% CI 51.4-57.0%): the interval excludes 50%, and the point estimate exceeds the 52.38% break-even. Random Forest shows the largest train-to-test gap (223 to 276), i.e. overfitting on ~4920 games. **Limitations:** only score-based features (no player, injury or line-movement data), one test season (wide confidence interval), tuning on one validation season. **Future work:** add odds-line features and player-level data, tune with walk-forward cross-validation across seasons, predict Over/Under directly as a classification task, and evaluate over more seasons. **Exploratory:** betting only when the model differs from the line by 3 or more points gives 57.3% on 597 games; this subset is small and was chosen after seeing results, so it is not evidence of a profitable strategy.

Reminder: these conclusions describe the synthetic demo data only; they must be re-read after regenerating with real data.